Kitty Numbers

Kitty the Cat, having heard of Leonardo Bonacci's life story (the inventor of the Fibonacci numbers) decides to invent her very own Kitty Numbers.
Kitty, wanting her name to be known by everyone, defines a sequence of numbers (which we refer to as the Kitty Numbers), as such:

$Kitty_1 = 11$

$Kitty_2 = 9$

$Kitty_3 = 20$

$Kitty_4 = 20$

$Kitty_5 = 25$

$Kitty_n = Kitty_{n-1} + Kitty_{n-2} + Kitty_{n-3} + Kitty_{n-4} + Kitty_{n-5}$

As an example, the 6_{th} term of the Kitty Numbers is
$\text{Kitty}_6 = 25 + 20 + 20 + 9 + 11 = 85$

To celebrate Kitty's achievement, her friend Rabbits the bunny wanted to throw a party and bake exactly $Kitty_x$ cookies, where $1 \leq x \leq N$. She knows that there will be exactly 3 guests attending the party, and she will *only* bake a number of cookies such that **all** the cookies can be evenly distributed among the 3 guests, leaving **no** cookies to waste.
Note that each cookie can only be eaten by a single guest, and cannot be shared among multiple guests.

You will be given the value of N, and your task is to help Rabbits find out how many values of $Kitty_x$ she could bake.

Constraints

- $2 \leq N \leq 10^{15}$

Sample Input

```
7
```

Sample Output

```
2
```

Explanation

We want to find out how many terms among the first 7 Kitty Numbers could be baked by Rabbits for her party.
The first 7 Kitty Numbers are:

```
11 9 20 20 25 85 159
```

And we observe that only the terms:

```
9 159
```

are possible numbers of cookies that Rabbits can bake.

For example, if Rabbits bakes 9 cookies, each of 3 guests will receive 3 cookies, with 0 cookies left over.
Similarly, if Rabbits bakes 159 cookies, each guest will receive 53 cookies, with 0 cookies left over.
Therefore, the expected output should be 2.

Solution

The first solution that comes to mind would be to generate the first N terms of the Kitty Numbers, and just count how many of them are divisible by 3. Depending on your implementation (and how long you leave your program running), this method would be fast enough to obtain anywhere between 30 and 70 points.

Below is one possible solution that scores 50 points. (It takes 5 seconds per test case for cases 1 to 5)

```
n = 59632
ans = 0
l = [11, 9, 20, 20, 25]

for i in range(n - 5):
    l.append(l[-1] + l[-2] + l[-3] + l[-4] + l[-5])

for x in l:
    if x % 3 == 0:
        ans += 1

print(ans)
```

However, coming up with the intended solution requires some knowledge of mathematics.

For convenience, let us introduce the remainder operator: $a \% b$

This operator returns the remainder of the first term a when divided by the second term b. For example, $10 \% 3 = 1$, because the remainder of 10 when divided by 3 is 1. Now, the problem is just finding the number of terms where $Kitty_x \% 3 = 0$, where $1 \leq x \leq N$.

Let us consider this result:
Let $f = a + b + c + d + e$
Then we have $f \% 3 = (a \% 3 + b \% 3 + c \% 3 + d \% 3 + e \% 3) \% 3$

For example, set $a = 11, b = 9, c = 20, d = 20, e = 25$, and we have `11 + 9 + 20 + 20 + 25 = 85`, which has the remainder `85 % 3 = 1`
But also,

```
(11 % 3 + 9 % 3 + 20 % 3 + 20 % 3 + 25 % 3) % 3
= (2 + 0 + 2 + 2 + 1) % 3
= 7 % 3
= 1
```

Which is also `85 % 3`, so the result holds. You might want to play around with a few more examples to convince yourself of this fact. This can be proven by writing each variable as $3q + r$.

So now we have a rather powerful result:
To get $Kitty_x \% 3$, we only need to consider

$Kitty_{x-1} \% 3, Kitty_{x-2} \% 3, Kitty_{x-3} \% 3, Kitty_{x-4} \% 3, Kitty_{x-5} \% 3$.
Everything can be done in remainders of 3!

Now, notice that the remainders of each term must be either 0, 1, or 2. Let us take a detour to examine the possible remainders for 5 consecutive Kitty Numbers, which will determine the remainder of the Kitty Number that follows.

Start of Detour

First, let us ask how many possible set of remainders we need to consider if $Kitty_x$ depends only on the remainder of the previous term, $Kitty_{x-1}$? That is, how many possible tuples of $Kitty_{x-1} \% 3$ are there?

```
1. 0
2. 1
3. 2
```

The answer would be 3.

And what if $Kitty_x$ depends on the remainders of the previous 2 terms? How many possible tuples of $Kitty_{x-1} \% 3, Kitty_{x-2} \% 3$ are there?

```
1. 0 0
2. 0 1
3. 0 2
4. 1 0
5. 1 1
6. 1 2
7. 2 0
8. 2 1
9. 2 2
```

There will be 9 tuples of remainders to consider.

If we continue investigating the case where it depends on the previous 3 terms, we would find there are 27 possible tuples of remainders to consider.

We can observe an interesting pattern here: whenever we add a term, we multiply the number of possible tuples by 3.

And so, if $Kitty_x$ depends on its 5 previous terms, the number of possible remainders tuples would be $3 * 3 * 3 * 3 * 3 = 243$

End of Detour

The number of possible remainders tuples is found to be only 243. Let us consider again the sequence of Kitty Numbers, but this time in remainders of 3.

```
Kitty Number:
11 9 20 20 25 85 159 309 598 1176 ...
Remainder:
2 0 2 2 1 1 0 0 1 0 ...
```

If we look at the 5-tuples (`2 0 2 2 1`, `0 2 2 1 1`, `2 2 1 1 0`, ...), we should expect there to be a cycle eventually (try to think why). Indeed

`2 0 2 2 1` $\rightarrow$ `1` 1st term

`0 2 2 1 1` $\rightarrow$ `0` 2nd term

`2 2 1 1 0` $\rightarrow$ `0` 3rd term

`2 1 1 0 0` $\rightarrow$ `1` 4th term

`1 1 0 0 1` $\rightarrow$ `0` 5th term

...

`1 2 0 2 2` $\rightarrow$ `1` 104th term

`2 0 2 2 1` $\rightarrow$ `1` 105th term (The same as the first one!)

Aha! Doing some simple calculations, we find the number of terms to be 104, and the number of terms that are divisible by 3 in this cycle is 35.

Below is one possible way to find that out.

```
remainders = []

a = 11 % 3
b = 9 % 3
```

```
c = 20 % 3
d = 20 % 3
e = 25 % 3

length = 0
terms_divisible_by_3 = 0

while (a, b, c, d, e) not in remainders:
    remainders.append((a, b, c, d, e))
    f = (a + b + c + d + e) % 3
    a, b, c, d, e = b, c, d, e, f

    length += 1
    if f % 3 == 0:
        terms_divisible_by_3 += 1

print((a, b, c, d, e))
print("The length of cycle is " + str(length))
print("The number of terms divisible by 3 is " + str(terms_divisible_by_3))
```

Output

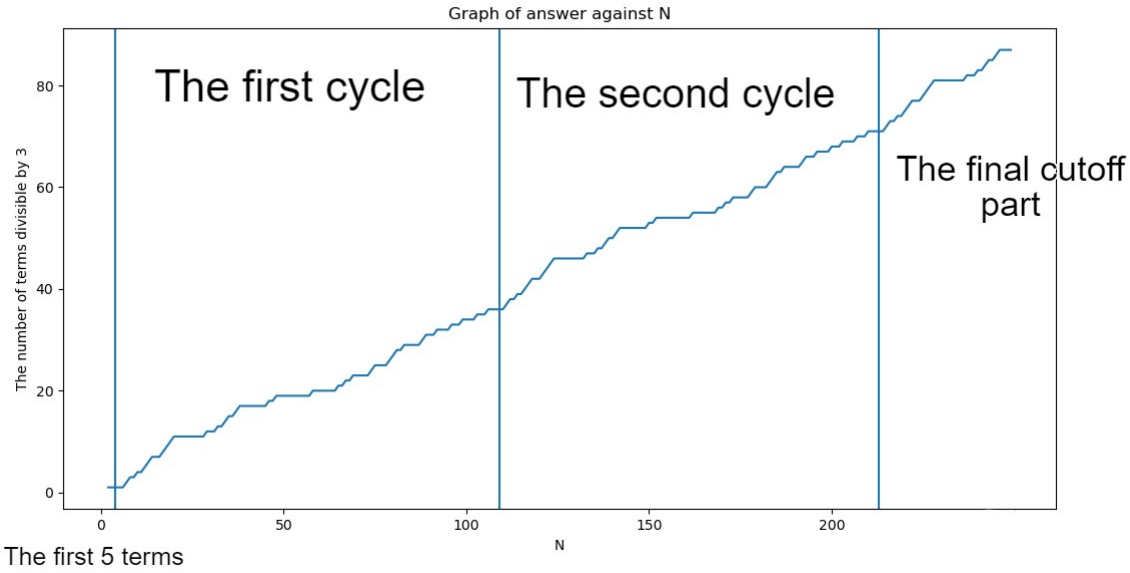
```
(2, 0, 2, 2, 1)
The length of cycle is 104
The number of terms divisible by 3 is 35
```

Note that the program will terminate quickly because there are at most 243 tuples to consider.

With all the information we have now, we can see that the N remainder terms of Kitty Numbers largely consists of these cycles repeating themselves.

Furthermore, because we know the number of terms in each cycle where the remainder is 0, we also know the number of terms in each cycle where they are divisible by 3. (Hint: They're the same. Can you see why?)

Thus, if we can count the number of cycles, we can arrive to a number that is close to the answer. What's left is to count the remaining part of the cycle - the part that has been cut off.



As seen above, we can split N into 3 parts:

- The first 5 terms, which contribute 1 to the answer
- Several complete cycles, each contributing 35 to the answer
- The final cycle that has been cut off

We can count the number of cycles by removing the first 5 elements from N, then dividing it by the number of terms in the cycle, namely 104.

As for the final part, we know the number of terms in it is fewer than 104, because it is shorter than a complete cycle. Therefore, we can just generate the remaining terms and increase the answer whenever we generate a term that is divisible by 3. The following is a complete solution in Python.

Implementation

```
n = 9210385091203523

ans = 1 # contribution of the first 5 terms
n -= 5
ans += ((n // 104) * 35) # contribution of each complete cycle
n %= 104

# start from the beginning of the cycle
a, b, c, d, e = 11, 9, 20, 20, 25
f = 0
```

```
for i in range(n):
    f = a + b + c + d + e

    # if term is divisible by 3, increment the answer
    if (f % 3 == 0):
        ans += 1
    a, b, c, d, e = b, c, d, e, f

print(ans)
```